

Lab Evaluation 2 – Dynamic Programming

Knapsack Family, LIS Family, LCS and Matrix Chain Multiplication

Muhtasim – CSE, University of Dhaka

oday

Contents

Introduction	2
PART A – 0/1 Knapsack Family	3
A.1 0/1 Knapsack (Bounded) – Core Problem	3
A.2 Subset Sum Problem	6
A.3 Equal Sum Partition (Partition Equal Subset Sum)	7
A.4 Count of Subsets with a Given Sum	8
A.5 Minimum Subset Sum Difference	9
A.6 Count Subsets with a Given Difference	10
A.7 Target Sum (LeetCode 494)	11
PART B – Longest Increasing Subsequence (LIS) Family	13
B.1 Longest Increasing Subsequence – Core Problem	13
B.2 Print / Reconstruct the LIS (Actual Sequence)	15
B.3 Number of Longest Increasing Subsequences (LeetCode 673)	16
B.4 Longest Non-Decreasing Subsequence	17
B.5 Longest Increasing Subsequence in a Circular Array	18
B.6 Longest Bitonic Subsequence	19
B.7 Longest Arithmetic Subsequence	20
B.8 Maximum Sum Increasing Subsequence	21
B.9 Largest Divisible Subset (LeetCode 368)	22
B.10 Russian Doll Envelopes (LeetCode 354)	23
B.11 Maximum Length of Pair Chain (LeetCode 646)	24
B.12 Delete and Earn (LeetCode 740)	25
B.13 Longest String Chain (LeetCode 1048)	26
PART C – More 0/1 / Unbounded Knapsack Family Problems	27
C.1 Coin Change (LeetCode 322)	27
C.2 Coin Change II (LeetCode 518)	29
C.3 Ones and Zeroes (LeetCode 474)	30
C.4 Last Stone Weight II (LeetCode 1049)	31
C.5 Last Stone Weight I (LeetCode 1046)	33
PART D – More Longest Increasing Subsequence Family Problems	34
D.1 Longest Continuous Increasing Subsequence (LeetCode 674)	34
D.2 Increasing Triplet Subsequence (LeetCode 334)	35
D.3 Wiggle Subsequence (LeetCode 376)	36
D.4 Minimum Number of Removals to Make Mountain Array (LeetCode 1671)	37
D.5 Best Team With No Conflicts (LeetCode 1626)	38

PART E – Classic 2-D / Interval DP: LCS and Matrix Chain Multiplication	39
E.1 Longest Common Subsequence (LCS)	39
E.2 Matrix Chain Multiplication (MCM)	41
PART F – Even More Knapsack Family Problems	43
F.1 House Robber (LeetCode 198)	43
F.2 House Robber II (LeetCode 213)	44
F.3 Perfect Squares (LeetCode 279)	45
F.4 Combination Sum IV (LeetCode 377)	46
F.5 Word Break (LeetCode 139)	47
PART G – Even More LIS Family Problems	48
G.1 Longest Increasing Path in a Matrix (LeetCode 329)	48
G.2 Count Number of Teams (LeetCode 1395)	49
Summary Table	50
Conclusion	51

Introduction

Dynamic Programming (DP) solves a problem by breaking it into overlapping sub-problems and reusing their answers instead of recomputing them. Every problem in this lab is solved using the **same three-step ladder**:

1. **Plain Recursion** – write the brute-force recursive solution first. Identify the *state* (the variables that change on each call) and the *choices* (what decisions we make at each step).
2. **Top-Down DP (Memoization)** – take the exact same recursion and cache the result of each state in an array/map (`dp[state] = ans`) so that a repeated state is computed only once.
3. **Bottom-Up DP (Tabulation)** – remove recursion entirely. Build a table iteratively from the smallest sub-problem up to the final answer, in the order the recursion would have unwound.

We apply this ladder in full detail to the two “parent” problems of this lab – **0/1 Knapsack** and **Longest Increasing Subsequence (LIS)** – because almost every other problem in the sheet is a small twist on one of these two patterns. For the remaining variant problems we explain the twist in plain language and give one clean, working implementation (usually bottom-up, since that is what you will use in practice/interviews).

Notation used throughout: n = size of input array, `dp[]` = table storing sub-problem answers, $O(\dots)$ = time/space complexity.

PART A – 0/1 Knapsack Family

A.1 0/1 Knapsack (Bounded) – Core Problem

Problem: Given n items, each with a `weight[i]` and a `value[i]`, and a knapsack of capacity W , choose a subset of items (each item used **at most once** – hence “0/1”) so that the total weight $\leq W$ and the total value is **maximized**.

Understanding

At every item we only have **two choices**:

- **Skip** the item -> move to the next item with the same capacity.
- **Take** the item (only if `weight[i] <= remaining capacity`) -> add `value[i]` and move to the next item with reduced capacity.

We take the **maximum** of these two choices. This “include / exclude” decision at every index is the heart of every problem in Part A.

State: (i, W) = index of the item we are considering, and the capacity left.

Step 1 – Plain Recursion

```
// f(i, W) = max value achievable using items [0..i] with capacity W
int f(int i, int W, vector<int>& wt, vector<int>& val) {
    // Base case: no items left to consider
    if (i == 0) {
        return (wt[0] <= W) ? val[0] : 0;
    }

    int notTake = f(i - 1, W, wt, val);           // skip item i
    int take = INT_MIN;
    if (wt[i] <= W)
        take = val[i] + f(i - 1, W - wt[i], wt, val); // take item i

    return max(take, notTake);
}

int knapsack(int W, vector<int>& wt, vector<int>& val) {
    int n = wt.size();
    return f(n - 1, W, wt, val);
}
```

Why it is slow: the same (i, W) pair is recomputed many times through different branches, giving exponential time $O(2^n)$.

Step 2 – Top-Down DP (Memoization)

We simply cache the answer for every (i, W) pair the first time it is computed.

```
int f(int i, int W, vector<int>& wt, vector<int>& val, vector<vector<int>>& dp) {
    if (i == 0) return (wt[0] <= W) ? val[0] : 0;

    if (dp[i][W] != -1) return dp[i][W]; // already solved -> reuse

    int notTake = f(i - 1, W, wt, val, dp);
    int take = INT_MIN;
```

```

    if (wt[i] <= W)
        take = val[i] + f(i - 1, W - wt[i], wt, val, dp);

    return dp[i][W] = max(take, notTake);
}

int knapsack(int W, vector<int>& wt, vector<int>& val) {
    int n = wt.size();
    vector<vector<int>> dp(n, vector<int>(W + 1, -1));
    return f(n - 1, W, wt, val, dp);
}

```

Complexity: $O(n * W)$ time, $O(n * W)$ space (table) + $O(n)$ recursion stack.

Step 3 – Bottom-Up DP (Tabulation)

We build $dp[i][w]$ iteratively, starting from $i = 0$ up to $i = n-1$, mirroring the recursion but with loops instead of function calls.

```

int knapsack(int W, vector<int>& wt, vector<int>& val) {
    int n = wt.size();
    vector<vector<int>> dp(n, vector<int>(W + 1, 0));

    // base row: only item 0 available
    for (int w = 0; w <= W; w++)
        dp[0][w] = (wt[0] <= w) ? val[0] : 0;

    for (int i = 1; i < n; i++) {
        for (int w = 0; w <= W; w++) {
            int notTake = dp[i - 1][w];
            int take = INT_MIN;
            if (wt[i] <= w)
                take = val[i] + dp[i - 1][w - wt[i]];
            dp[i][w] = max(take, notTake);
        }
    }
    return dp[n - 1][W];
}

```

Space Optimization (1-D array)

Since row i only depends on row $i-1$, we can collapse the table to a single array of size $W+1$, **looping the capacity from right to left** (so we don't overwrite a value we still need from the "previous row"):

```

int knapsack(int W, vector<int>& wt, vector<int>& val) {
    int n = wt.size();
    vector<int> dp(W + 1, 0);

    for (int w = wt[0]; w <= W; w++) dp[w] = val[0];

    for (int i = 1; i < n; i++) {
        for (int w = W; w >= wt[i]; w--) { // right to left!
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }
}

```

```
    return dp[W];  
}
```

Complexity: $O(n * W)$ time, $O(W)$ space.

Every problem below is this exact same “take / not take” recursion applied to a slightly different question, so we will move faster and only show the clean, final code for each.

A.2 Subset Sum Problem

Problem: Given an array of n positive integers and a target **sum**, determine whether **any subset** of the array adds up exactly to **sum**.

Understanding: This is 0/1 Knapsack where “weight = value = $\text{arr}[i]$ ” and instead of maximizing value we only ask a yes/no question: “*can capacity **sum** be exactly filled?*” $\text{dp}[i][s] = \text{true}$ if some subset of the first i elements sums to s .

Recurrence: $\text{dp}[i][s] = \text{dp}[i-1][s] \text{ OR } (s \geq \text{arr}[i] \text{ AND } \text{dp}[i-1][s-\text{arr}[i]])$

```
bool subsetSum(vector<int>& arr, int sum) {
    int n = arr.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(sum + 1, false));

    for (int i = 0; i <= n; i++) dp[i][0] = true;    // sum 0 always possible (empty subset)

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= sum; s++) {
            dp[i][s] = dp[i - 1][s];                // don't take arr[i-1]
            if (arr[i - 1] <= s)
                dp[i][s] = dp[i][s] || dp[i - 1][s - arr[i - 1]]; // take it
        }
    }
    return dp[n][sum];
}
```

Complexity: $O(n * \text{sum})$ time and space.

A.3 Equal Sum Partition (Partition Equal Subset Sum)

Problem: Given an array, determine if it can be split into **two subsets with equal sum**.

Understanding: Let **total** be the sum of the whole array. If **total** is odd, splitting into two equal halves is impossible immediately. Otherwise the question reduces exactly to **Subset Sum** with **target** = **total** / 2: if one half can be built, the remaining elements automatically form the other equal half.

```
bool canPartition(vector<int>& arr) {
    int total = accumulate(arr.begin(), arr.end(), 0);
    if (total % 2 != 0) return false;
    return subsetSum(arr, total / 2);    // reuse function from A.2
}
```

Complexity: $O(n * \text{total})$ time.

A.4 Count of Subsets with a Given Sum

Problem: Count **how many** subsets of the array sum exactly to a given **sum** (not just whether one exists).

Understanding: Same recursion as Subset Sum, but instead of OR-ing booleans we **add up counts** from the “take” and “not take” branches.

```
int countSubsets(vector<int>& arr, int sum) {
    int n = arr.size();
    vector<vector<int>> dp(n + 1, vector<int>(sum + 1, 0));

    for (int i = 0; i <= n; i++) dp[i][0] = 1;    // one way to make sum 0: pick nothing

    for (int i = 1; i <= n; i++) {
        for (int s = 0; s <= sum; s++) {
            int notTake = dp[i - 1][s];
            int take = (arr[i - 1] <= s) ? dp[i - 1][s - arr[i - 1]] : 0;
            dp[i][s] = notTake + take;
        }
    }
    return dp[n][sum];
}
```

Note (zeros in the array): if the array can contain 0s, each zero doubles the count of every subset (it can be freely included or excluded), so the base case / recurrence above must be adjusted ($dp[i][0]$ is no longer simply 1 for every i) – this exact subtlety shows up in LeetCode’s *Target Sum* below.

Complexity: $O(n * \text{sum})$ time and space.

A.5 Minimum Subset Sum Difference

Problem: Split the array into two subsets S1 and S2 such that $|\text{sum}(S1) - \text{sum}(S2)|$ is **minimized**.

Understanding: Let *total* be the array sum. If S1 has sum *s*, then S2 has sum *total - s*, and the difference is $|\text{total} - 2s|$. We use the **Subset Sum DP** (A.2) to find *every achievable sum s* in the range $[0, \text{total}/2]$, then pick the *s* closest to $\text{total}/2$ – this minimizes $|\text{total} - 2s|$.

```
int minSubsetSumDifference(vector<int>& arr) {
    int n = arr.size();
    int total = accumulate(arr.begin(), arr.end(), 0);

    vector<vector<bool>> dp(n + 1, vector<bool>(total + 1, false));
    for (int i = 0; i <= n; i++) dp[i][0] = true;

    for (int i = 1; i <= n; i++)
        for (int s = 1; s <= total; s++) {
            dp[i][s] = dp[i - 1][s];
            if (arr[i - 1] <= s)
                dp[i][s] = dp[i][s] || dp[i - 1][s - arr[i - 1]];
        }

    int minDiff = INT_MAX;
    for (int s = 0; s <= total / 2; s++) {
        if (dp[n][s]) // sum s is achievable
            minDiff = min(minDiff, total - 2 * s);
    }
    return minDiff;
}
```

Complexity: $O(n * \text{total})$ time and space.

A.6 Count Subsets with a Given Difference

Problem: Count the number of ways to split the array into two subsets S_1 , S_2 such that $\text{sum}(S_1) - \text{sum}(S_2) = \text{diff}$.

Understanding: Two equations, two unknowns: $S_1 + S_2 = \text{total}$ and $S_1 - S_2 = \text{diff} \Rightarrow S_1 = (\text{total} + \text{diff}) / 2$.

So this becomes an instance of **Count of Subsets with a Given Sum** (A.4), where the target sum is $(\text{total} + \text{diff}) / 2$. (If $\text{total} + \text{diff}$ is odd, or $\text{diff} > \text{total}$, the answer is 0 – no valid split exists.)

```
int countSubsetsWithDifference(vector<int>& arr, int diff) {
    int total = accumulate(arr.begin(), arr.end(), 0);
    if ((total + diff) % 2 != 0 || total < diff) return 0;
    int target = (total + diff) / 2;
    return countSubsets(arr, target);    // reuse function from A.4
}
```

Complexity: $O(n * \text{total})$ time.

A.7 Target Sum (LeetCode 494)

Link: <https://leetcode.com/problems/target-sum/>

Problem: Given an array `nums` and integer `target`, assign a + or - sign in front of each number so that the resulting expression equals `target`. Count the number of ways to do this.

Understanding: This is *exactly* **Count Subsets with a Given Difference** (A.6) in disguise: the numbers that get a + form subset P (positive), the numbers that get a - form subset N (negative), and we need $\text{sum}(P) - \text{sum}(N) = \text{target}$, which is precisely $\text{diff} = \text{target}$.

Step 1 – Plain Recursion

```
int f(int i, int currSum, vector<int>& nums, int target) {
    if (i == nums.size())
        return (currSum == target) ? 1 : 0;

    int add = f(i + 1, currSum + nums[i], nums, target);    // '+' sign
    int subtract = f(i + 1, currSum - nums[i], nums, target); // '-' sign
    return add + subtract;
}

int findTargetSumWays(vector<int>& nums, int target) {
    return f(0, 0, nums, target);
}
```

Step 2 – Top-Down DP (Memoization)

`currSum` can be negative, so we offset it by `total` (the sum of all elements) to use it safely as an array index.

```
int f(int i, int currSum, vector<int>& nums, int target, int total,
    vector<vector<int>>& dp) {
    if (i == nums.size())
        return (currSum == target) ? 1 : 0;

    int idx = currSum + total;    // shift into [0, 2*total]
    if (dp[i][idx] != -1) return dp[i][idx];

    int add = f(i + 1, currSum + nums[i], nums, target, total, dp);
    int subtract = f(i + 1, currSum - nums[i], nums, target, total, dp);

    return dp[i][idx] = add + subtract;
}

int findTargetSumWays(vector<int>& nums, int target) {
    int total = accumulate(nums.begin(), nums.end(), 0);
    if (abs(target) > total) return 0;    // impossible
    vector<vector<int>> dp(nums.size(), vector<int>(2 * total + 1, -1));
    return f(0, 0, nums, target, total, dp);
}
```

Complexity: $O(n * \text{total})$ time and space.

Step 3 – Bottom-Up DP (via Count Subsets with Given Difference)

Using the transformation from A.6: $\text{target} = |P| - |N|$, $\text{total} = |P| + |N| \Rightarrow |P| = (\text{total} + \text{target}) / 2$. This turns the whole problem into one call of **Count of Subsets with a Given Sum**.

```

int findTargetSumWays(vector<int>& nums, int target) {
    int total = accumulate(nums.begin(), nums.end(), 0);
    if (abs(target) > total || (total + target) % 2 != 0) return 0;

    int s = (total + target) / 2;    // required sum of the "positive" subset
    int n = nums.size();

    vector<int> dp(s + 1, 0);
    dp[0] = 1;
    for (int i = 0; i < n; i++) {
        for (int w = s; w >= nums[i]; w--) {
            dp[w] += dp[w - nums[i]];
        }
    }
    return dp[s];
}

```

Complexity: $O(n * \text{total})$ time, $O(\text{total})$ space – much faster than plain recursion's $O(2^n)$.

PART B – Longest Increasing Subsequence (LIS) Family

B.1 Longest Increasing Subsequence – Core Problem

Link: <https://leetcode.com/problems/longest-increasing-subsequence/>

Problem: Given an array `nums`, find the length of the **longest strictly increasing subsequence** (elements need not be contiguous, but their relative order must be preserved and each next element must be strictly greater than the previous).

Understanding

Define $f(i)$ = length of the longest increasing subsequence that **ends exactly at index i** . For every index i , we look at all previous indices $j < i$: if `nums[j] < nums[i]`, then index i could extend the subsequence ending at j . We take the best such extension:

$f(i) = 1 + \max(f(j))$ for all $j < i$ with `nums[j] < nums[i]` (or just 1 if no such j exists – the element on its own).

The final answer is $\max(f(i))$ over all i , because the LIS can end anywhere.

Step 1 – Plain Recursion

```
// f(i) = length of LIS ending at index i
int f(int i, vector<int>& nums) {
    int best = 1; // the element alone is a subsequence of length 1
    for (int j = 0; j < i; j++) {
        if (nums[j] < nums[i]) {
            best = max(best, 1 + f(j, nums));
        }
    }
    return best;
}

int lengthOfLIS(vector<int>& nums) {
    int n = nums.size(), ans = 1;
    for (int i = 0; i < n; i++)
        ans = max(ans, f(i, nums));
    return ans;
}
```

Complexity: exponential – $f(i)$ is recomputed repeatedly for the same i from different callers.

Step 2 – Top-Down DP (Memoization)

```
int f(int i, vector<int>& nums, vector<int>& dp) {
    if (dp[i] != -1) return dp[i];

    int best = 1;
    for (int j = 0; j < i; j++)
        if (nums[j] < nums[i])
            best = max(best, 1 + f(j, nums, dp));

    return dp[i] = best;
}
```

```
int lengthOfLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, -1);
    int best = 1;
    for (int i = 0; i < n; i++)
        best = max(best, f(i, nums, dp));
    return best;
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

Step 3 – Bottom-Up DP (Tabulation)

```
int lengthOfLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, 1);           // dp[i] = LIS length ending at i, min length is 1

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i])
                dp[i] = max(dp[i], 1 + dp[j]);
        }
    }
    return *max_element(dp.begin(), dp.end());
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

Step 4 – Optimal $O(n \log n)$ Approach (Patience Sorting)

Maintain an array `tails`, where `tails[k]` = the smallest possible **tail value** of an increasing subsequence of length `k+1` found so far. For each new number, binary-search for its position in `tails` and either extend `tails` or replace an entry – this keeps `tails` sorted and as “loose” as possible so future numbers have the best chance to extend it.

```
int lengthOfLIS(vector<int>& nums) {
    vector<int> tails;
    for (int x : nums) {
        auto it = lower_bound(tails.begin(), tails.end(), x); // first tail >= x
        if (it == tails.end()) tails.push_back(x);           // extend LIS
        else *it = x;                                         // replace to keep tails minimal
    }
    return tails.size();
}
```

Complexity: $O(n \log n)$ time, $O(n)$ space. Note: `tails` itself is **not** a valid LIS – only its *length* is guaranteed correct.

B.2 Print / Reconstruct the LIS (Actual Sequence)

Problem: Not just the length, but print one valid Longest Increasing Subsequence.

Understanding: Extend the bottom-up tabulation (B.1, Step 3) with a `parent[]` array. Whenever we update `dp[i] = 1 + dp[j]`, we also record `parent[i] = j` – i.e. “index `i` was reached by extending the subsequence that ended at `j`”. At the end we find the index with the maximum `dp[]` value and walk backwards through `parent[]`.

```
vector<int> printLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, 1), parent(n);
    iota(parent.begin(), parent.end(), 0);    // parent[i] = i initially (points to itself)

    int lastIdx = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                parent[i] = j;                // remember how we got here
            }
            if (dp[i] > dp[lastIdx]) lastIdx = i;
        }
    }

    // walk backwards from lastIdx using parent[] to rebuild the sequence
    vector<int> lis;
    while (parent[lastIdx] != lastIdx) {
        lis.push_back(nums[lastIdx]);
        lastIdx = parent[lastIdx];
    }
    lis.push_back(nums[lastIdx]);
    reverse(lis.begin(), lis.end());
    return lis;
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

B.3 Number of Longest Increasing Subsequences (LeetCode 673)

Link: <https://leetcode.com/problems/number-of-longest-increasing-subsequence/>

Problem: Find **how many** distinct increasing subsequences achieve the maximum LIS length (not just the length itself).

Understanding: Keep two arrays alongside each other:

- `len[i]` = length of the LIS ending at `i` (same as `dp[i]` before).
- `cnt[i]` = number of LIS's of that best length ending at `i`.

For each `j < i` with `nums[j] < nums[i]`:

- If `len[j] + 1 > len[i]` -> we found a **strictly longer** path to `i`, so reset: `len[i] = len[j] + 1`, `cnt[i] = cnt[j]`.
- If `len[j] + 1 == len[i]` -> we found **another way** to achieve the same best length, so accumulate: `cnt[i] += cnt[j]`.

Finally, sum `cnt[i]` over every index `i` whose `len[i]` equals the overall maximum length.

```
int findNumberOfLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> len(n, 1), cnt(n, 1);

    int maxLen = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                if (len[j] + 1 > len[i]) {
                    len[i] = len[j] + 1;
                    cnt[i] = cnt[j];           // fresh best path, reset count
                } else if (len[j] + 1 == len[i]) {
                    cnt[i] += cnt[j];         // another equally-good path
                }
            }
        }
        maxLen = max(maxLen, len[i]);
    }

    int total = 0;
    for (int i = 0; i < n; i++)
        if (len[i] == maxLen) total += cnt[i];
    return total;
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

B.4 Longest Non-Decreasing Subsequence

Problem: Same as LIS, but equal consecutive values are allowed ($\text{nums}[j] \leq \text{nums}[i]$ instead of strictly $<$).

Understanding: Literally a **one-character change** to B.1 – flip the comparison from $<$ to $\leq$, since we now allow the subsequence to stay flat as well as increase.

```
int longestNonDecreasingSubsequence(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (nums[j] <= nums[i])           // <= instead of <
                dp[i] = max(dp[i], 1 + dp[j]);
    return *max_element(dp.begin(), dp.end());
}
```

Complexity: $O(n^2)$ time.

B.5 Longest Increasing Subsequence in a Circular Array

Problem: The array is **circular** (the element after the last one wraps around to index 0). Find the LIS considering this circular property, i.e. a subsequence may “wrap” once around the end of the array.

Understanding: A simple, standard trick for circular problems is to **duplicate the array** (concatenate it with itself), then run ordinary LIS logic on the doubled array, but restrict any subsequence to a window of at most n original elements (i.e. don't reuse the same original index twice). A clean way: run LIS on `nums + nums` (length $2n$) using indices, but only count a subsequence starting at original index i and use j in $[i, i+n)$.

```
int circularLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> doubled(nums);
    doubled.insert(doubled.end(), nums.begin(), nums.end()); // nums + nums, size 2n

    int ans = 1;
    // try every starting rotation point as the beginning of the subsequence
    for (int start = 0; start < n; start++) {
        vector<int> dp(2 * n, 1);
        for (int i = start; i < start + n; i++) {
            for (int j = start; j < i; j++) {
                if (doubled[j] < doubled[i])
                    dp[i] = max(dp[i], 1 + dp[j]);
            }
            ans = max(ans, dp[i]);
        }
    }
    return ans;
}
```

Complexity: $O(n^3)$ (n starting points $\times O(n^2)$ LIS each) – acceptable for small/medium n . For large n , the $O(n \log n)$ technique can be adapted per rotation to bring this down to $O(n^2 \log n)$.

B.6 Longest Bitonic Subsequence

Problem: Find the length of the longest subsequence that first **strictly increases**, then **strictly decreases** (a “mountain” shape). Either the increasing or decreasing part may be empty-length-1 at the peak.

Understanding: Compute two arrays using ordinary LIS logic:

- `inc[i]` = length of the LIS **ending** at index `i` (increasing, left to right).
- `dec[i]` = length of the LIS **starting** at index `i` when the array is scanned **right to left** (equivalently, the longest strictly decreasing subsequence starting at `i` going left to right).

For every index `i` (treated as the “peak” of the mountain), the bitonic length through it is `inc[i] + dec[i] - 1` (subtracting 1 because `nums[i]` is counted in both halves). The answer is the maximum over all `i`.

```
int longestBitonicSubsequence(vector<int>& nums) {
    int n = nums.size();
    vector<int> inc(n, 1), dec(n, 1);

    // inc[i]: LIS ending at i, scanning left to right
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (nums[j] < nums[i])
                inc[i] = max(inc[i], 1 + inc[j]);

    // dec[i]: LIS starting at i, scanning right to left (i.e. longest decreasing run from i)
    for (int i = n - 1; i >= 0; i--)
        for (int j = n - 1; j > i; j--)
            if (nums[j] < nums[i])
                dec[i] = max(dec[i], 1 + dec[j]);

    int ans = 0;
    for (int i = 0; i < n; i++)
        ans = max(ans, inc[i] + dec[i] - 1);
    return ans;
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

B.7 Longest Arithmetic Subsequence

Problem: Find the length of the longest subsequence such that the difference between consecutive elements is constant (an arithmetic progression), e.g. [3, 6, 9, 12].

Understanding: Unlike plain LIS, the state now needs the **common difference** as well, because “increasing” isn’t enough – the *gap* must be consistent. Define $dp[i][d]$ = length of the longest arithmetic subsequence ending at index i with common difference d . For every pair $j < i$, the difference is $d = \text{nums}[i] - \text{nums}[j]$; index i can extend whatever arithmetic subsequence ended at j with that *same* difference d :

$dp[i][d] = dp[j][d] + 1$, defaulting to 2 if no such subsequence at j existed yet (the pair (j, i) itself forms a length-2 arithmetic subsequence).

We use a hash map per index (d can be negative and unbounded) instead of a fixed array.

```
int longestArithSeqLength(vector<int>& nums) {
    int n = nums.size();
    vector<unordered_map<int, int>> dp(n);    // dp[i][d] = length ending at i with diff d
    int ans = 1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            int d = nums[i] - nums[j];
            int prevLen = dp[j].count(d) ? dp[j][d] : 1;    // at least the pair (j,i)
            dp[i][d] = max(dp[i][d], prevLen + 1);
            ans = max(ans, dp[i][d]);
        }
    }
    return ans;
}
```

Complexity: $O(n^2)$ time and space (amortized, thanks to hash maps).

B.8 Maximum Sum Increasing Subsequence

Problem: Instead of maximizing the *length* of an increasing subsequence, maximize the **sum** of its elements.

Understanding: Identical structure to B.1, but the DP value stores accumulated **sum** instead of **count**.
 $dp[i]$ = maximum sum of an increasing subsequence ending at index i , initialized to $nums[i]$ itself (the element alone).

```
int maxSumIncreasingSubsequence(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(nums);           // dp[i] starts as nums[i] (subsequence of just itself)

    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (nums[j] < nums[i])
                dp[i] = max(dp[i], dp[j] + nums[i]);

    return *max_element(dp.begin(), dp.end());
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

B.9 Largest Divisible Subset (LeetCode 368)

Link: <https://leetcode.com/problems/largest-divisible-subset/>

Problem: Given a set of distinct integers, find the largest subset such that for every pair (a, b) in it, either $a \% b == 0$ or $b \% a == 0$.

Understanding: Sort the array first. Once sorted, if $a < b$ and we only require $b \% a == 0$, this becomes *identical to LIS*, except the comparison $\text{nums}[j] < \text{nums}[i]$ is replaced with $\text{nums}[i] \% \text{nums}[j] == 0$. Sorting guarantees that if a divides b and both divide c, then $a < b < c$, so the “chain” property lines up with array order – exactly like the increasing property in LIS. We also keep a `parent[]` array to reconstruct the actual subset (like B.2).

```
vector<int> largestDivisibleSubset(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    int n = nums.size();
    vector<int> dp(n, 1), parent(n);
    iota(parent.begin(), parent.end(), 0);

    int lastIdx = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[i] % nums[j] == 0 && dp[j] + 1 > dp[i]) { // divisibility instead of "<"
                dp[i] = dp[j] + 1;
                parent[i] = j;
            }
        }
        if (dp[i] > dp[lastIdx]) lastIdx = i;
    }

    vector<int> result;
    while (parent[lastIdx] != lastIdx) {
        result.push_back(nums[lastIdx]);
        lastIdx = parent[lastIdx];
    }
    result.push_back(nums[lastIdx]);
    reverse(result.begin(), result.end());
    return result;
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

B.10 Russian Doll Envelopes (LeetCode 354)

Link: <https://leetcode.com/problems/russian-doll-envelopes/>

Problem: Envelopes have (width, height). One envelope fits inside another only if **both** its width and height are strictly smaller. Find the maximum number of envelopes that can be nested (like Russian dolls) in a chain.

Understanding: This is **LIS in 2 dimensions**, reduced to 1 dimension with a sorting trick:

1. **Sort** envelopes by width ascending. For envelopes with the **same width**, sort by height **descending**.
2. Now run **plain LIS ($O(n \log n)$)** on just the **heights**.

Why sort ties by descending height? Because two envelopes of equal width can never nest inside each other (width must be strictly smaller). Sorting equal-width groups by descending height prevents the LIS from mistakenly picking two same-width envelopes as if they were nestable – since heights of same-width envelopes now appear in decreasing order, LIS (which needs *strictly increasing*) can pick at most one from each width group.

```
int maxEnvelopes(vector<vector<int>>& envelopes) {
    sort(envelopes.begin(), envelopes.end(), [](vector<int>& a, vector<int>& b) {
        if (a[0] == b[0]) return a[1] > b[1];    // same width -> height descending
        return a[0] < b[0];                      // otherwise width ascending
    });

    vector<int> tails;    //  $O(n \log n)$  LIS on heights (see B.1, Step 4)
    for (auto& env : envelopes) {
        int h = env[1];
        auto it = lower_bound(tails.begin(), tails.end(), h);
        if (it == tails.end()) tails.push_back(h);
        else *it = h;
    }
    return tails.size();
}
```

Complexity: $O(n \log n)$ time (sort + patience-sort LIS), $O(n)$ space.

B.11 Maximum Length of Pair Chain (LeetCode 646)

Link: <https://leetcode.com/problems/maximum-length-of-pair-chain/>

Problem: Given pairs (a, b) with $a < b$, a pair (c, d) can follow (a, b) if $b < c$. Find the longest chain that can be formed (pairs can be reordered).

Understanding: This looks like LIS on pairs ($O(n^2)$), but because chaining only depends on “does the previous pair’s second value end before this pair’s first value starts”, a **greedy** approach after sorting by the second value achieves the optimum in $O(n \log n)$ – this is the same idea as the classic “activity selection” / interval scheduling problem. We show both.

DP Approach (LIS-style, $O(n^2)$)

```
int findLongestChainDP(vector<vector<int>>& pairs) {
    sort(pairs.begin(), pairs.end());    // sort by first element
    int n = pairs.size();
    vector<int> dp(n, 1);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (pairs[j][1] < pairs[i][0])    // chainable: prev end < curr start
                dp[i] = max(dp[i], 1 + dp[j]);

    return *max_element(dp.begin(), dp.end());
}
```

Greedy Approach (Optimal, $O(n \log n)$)

Sort by the **second** value of each pair; always pick the next pair whose start is after the current chain’s last end (exactly like the classroom “maximum non-overlapping intervals” problem).

```
int findLongestChainGreedy(vector<vector<int>>& pairs) {
    sort(pairs.begin(), pairs.end(),
         [](vector<int>& a, vector<int>& b) { return a[1] < b[1]; });    // sort by end value

    int count = 0, currEnd = INT_MIN;
    for (auto& p : pairs) {
        if (p[0] > currEnd) {    // chainable
            count++;
            currEnd = p[1];
        }
    }
    return count;
}
```

Complexity: DP: $O(n^2)$. Greedy: $O(n \log n)$ – preferred in practice.

B.12 Delete and Earn (LeetCode 740)

Link: <https://leetcode.com/problems/delete-and-earn/>

Problem: Given `nums`, picking a number `x` earns `x` points but **deletes every occurrence of `x-1` and `x+1`** from the array (they become unusable). Maximize total points earned.

Understanding: This is **not** an LIS variant – it is a disguised version of **House Robber**, which itself is closely related to the “take/not take” idea from 0/1 Knapsack (A.1). Steps:

1. Build a `points[]` array indexed by value: `points[v] = v * (number of times v appears in nums)`.
2. Because choosing value `v` forbids both neighbors `v-1` and `v+1`, adjacent *values* behave like adjacent *houses* in House Robber – you cannot take two consecutive values.
3. Run the classic House Robber DP over `points[]`.

```
int deleteAndEarn(vector<int>& nums) {
    int maxVal = *max_element(nums.begin(), nums.end());
    vector<int> points(maxVal + 1, 0);
    for (int x : nums) points[x] += x;    // bucket + accumulate value

    // House Robber DP: can't take points[i] and points[i-1] together
    int n = maxVal + 1;
    vector<int> dp(n, 0);
    dp[0] = points[0];
    if (n > 1) dp[1] = max(points[0], points[1]);

    for (int i = 2; i < n; i++)
        dp[i] = max(dp[i - 1], dp[i - 2] + points[i]);    // skip i, or take i + best from i-2

    return dp[n - 1];
}
```

Complexity: $O(n + \text{maxVal})$ time, $O(\text{maxVal})$ space.

B.13 Longest String Chain (LeetCode 1048)

Link: <https://leetcode.com/problems/longest-string-chain/>

Problem: A word A is a **predecessor** of word B if you can insert exactly one letter into A (anywhere) to get B. Given a list of words, find the length of the longest chain of words $\text{word1} \rightarrow \text{word2} \rightarrow \dots \rightarrow \text{wordk}$ where each is a predecessor of the next.

Understanding: Once again, this is **LIS**, but with two twists:

1. **Sort words by length** first – a predecessor is always exactly 1 character shorter, so a valid chain must appear in increasing length order (this plays the role that “sorted array” plays for regular LIS).
2. The comparison $\text{nums}[j] < \text{nums}[i]$ is replaced with a helper function `isPredecessor(word[j], word[i])` that checks whether `word[j]` can become `word[i]` by inserting one letter.

```
bool isPredecessor(string& shorter, string& longer) {
    if (longer.size() != shorter.size() + 1) return false;
    int i = 0, j = 0;
    bool usedSkip = false;
    while (i < shorter.size() && j < longer.size()) {
        if (shorter[i] == longer[j]) {
            i++; j++;
        } else {
            if (usedSkip) return false; // already used our one allowed insertion
            usedSkip = true;
            j++; // skip a character in the longer word
        }
    }
    return true;
}

int longestStrChain(vector<string>& words) {
    sort(words.begin(), words.end(),
        [](const string& a, const string& b) { return a.size() < b.size(); });

    int n = words.size();
    vector<int> dp(n, 1);
    int ans = 1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (isPredecessor(words[j], words[i]))
                dp[i] = max(dp[i], 1 + dp[j]);
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

Complexity: $O(n^2 * L)$ time (L = max word length for the predecessor check), $O(n)$ space.

PART C – More 0/1 / Unbounded Knapsack Family Problems

C.1 Coin Change (LeetCode 322)

Link: <https://leetcode.com/problems/coin-change/>

Problem: Given coin denominations `coins[]` (unlimited supply of each) and a target `amount`, find the **minimum number of coins** needed to make up `amount`. Return `-1` if impossible.

Understanding: This is **Unbounded Knapsack** – unlike A.1 where each item is used at most once, here each coin can be reused any number of times. The only change from 0/1 Knapsack's recursion is: when we “take” a coin, we stay on the **same index** `i` (not `i-1`), because the coin is still available again.

State: (`i`, `amount`). **Choices:** skip coin `i`, or use coin `i` (if it fits) and stay at index `i`.

Step 1 – Plain Recursion

```
int f(int i, int amount, vector<int>& coins) {
    if (i == 0) {
        return (amount % coins[0] == 0) ? amount / coins[0] : INT_MAX / 2;
    }
    int notTake = f(i - 1, amount, coins);
    int take = INT_MAX / 2;
    if (coins[i] <= amount)
        take = 1 + f(i, amount - coins[i], coins);    // stay at i -- coin reusable
    return min(take, notTake);
}

int coinChange(vector<int>& coins, int amount) {
    int n = coins.size();
    int ans = f(n - 1, amount, coins);
    return (ans >= INT_MAX / 2) ? -1 : ans;
}
```

Step 2 – Top-Down DP (Memoization)

```
int f(int i, int amount, vector<int>& coins, vector<vector<int>>& dp) {
    if (i == 0)
        return (amount % coins[0] == 0) ? amount / coins[0] : INT_MAX / 2;
    if (dp[i][amount] != -1) return dp[i][amount];

    int notTake = f(i - 1, amount, coins, dp);
    int take = INT_MAX / 2;
    if (coins[i] <= amount)
        take = 1 + f(i, amount - coins[i], coins, dp);

    return dp[i][amount] = min(take, notTake);
}

int coinChange(vector<int>& coins, int amount) {
    int n = coins.size();
    vector<vector<int>> dp(n, vector<int>(amount + 1, -1));
    int ans = f(n - 1, amount, coins, dp);
    return (ans >= INT_MAX / 2) ? -1 : ans;
}
```

Step 3 – Bottom-Up DP (Tabulation, space-optimized to 1-D)

```
int coinChange(vector<int>& coins, int amount) {  
    vector<int> dp(amount + 1, INT_MAX / 2);  
    dp[0] = 0; // 0 coins needed to make amount 0  
  
    for (int coin : coins) {  
        for (int a = coin; a <= amount; a++) { // left to right: coin can repeat  
            dp[a] = min(dp[a], 1 + dp[a - coin]);  
        }  
    }  
    return (dp[amount] >= INT_MAX / 2) ? -1 : dp[amount];  
}
```

Notice the inner loop goes **left to right** here (coin reusable), while 0/1 Knapsack's space-optimized loop went **right to left** (item used once). This left-to-right vs right-to-left direction is *the* signature difference between Unbounded and 0/1 Knapsack.

Complexity: $O(n * \text{amount})$ time, $O(\text{amount})$ space.

C.2 Coin Change II (LeetCode 518)

Link: <https://leetcode.com/problems/coin-change-ii/>

Problem: Same coins (unlimited supply), but now count the **number of distinct combinations** of coins that add up to **amount** (order doesn't matter).

Understanding: Unbounded Knapsack again, but counting ways (like A.4) instead of minimizing count. Same left-to-right reuse trick as C.1, applied to a “sum of ways” recurrence instead of “min of counts”.

```
int change(int amount, vector<int>& coins) {
    vector<int> dp(amount + 1, 0);
    dp[0] = 1;    // one way to make amount 0: choose nothing

    for (int coin : coins) {                // outer loop over coins avoids counting
        for (int a = coin; a <= amount; a++) { // permutations as separate combinations
            dp[a] += dp[a - coin];
        }
    }
    return dp[amount];
}
```

Why coins must be the outer loop: if **amount** were the outer loop instead, the same combination (e.g. 1+2 and 2+1) would be counted twice as different *orderings*. Looping coins on the outside guarantees each **combination** (unordered) is counted exactly once.

Complexity: $O(n * \text{amount})$ time, $O(\text{amount})$ space.

C.3 Ones and Zeroes (LeetCode 474)

Link: <https://leetcode.com/problems/ones-and-zeroes/>

Problem: Given a list of binary strings and integers m , n , find the size of the largest subset such that the subset contains **at most m zeros and n ones in total**.

Understanding: This is 0/1 Knapsack with **two capacity dimensions** instead of one – each string “costs” some number of zeros and some number of ones (like two different weight types), and we still choose to take or skip each string at most once. State becomes $(i, \text{zerosLeft}, \text{onesLeft})$.

```
int findMaxForm(vector<string>& strs, int m, int n) {
    int len = strs.size();
    vector<vector<vector<int>>> dp(
        len + 1, vector<vector<int>>(m + 1, vector<int>(n + 1, 0)));

    for (int i = 1; i <= len; i++) {
        int zeros = count(strs[i - 1].begin(), strs[i - 1].end(), '0');
        int ones = strs[i - 1].size() - zeros;

        for (int z = 0; z <= m; z++) {
            for (int o = 0; o <= n; o++) {
                int notTake = dp[i - 1][z][o];
                int take = (zeros <= z && ones <= o)
                    ? 1 + dp[i - 1][z - zeros][o - ones]
                    : 0;
                dp[i][z][o] = max(notTake, take);
            }
        }
    }
    return dp[len][m][n];
}
```

Complexity: $O(\text{len} * m * n)$ time and space – the exact same “take / not take” skeleton as A.1, just with two capacities carried together.

C.4 Last Stone Weight II (LeetCode 1049)

Link: <https://leetcode.com/problems/last-stone-weight-ii/>

Problem: You may repeatedly choose **any** two stones and smash them: if equal weight both vanish, otherwise the smaller vanishes and the larger becomes $|x - y|$. Choosing the smashing order freely, find the **minimum possible weight** of the stone left at the end (0 if none remain).

Understanding: No matter what order stones are smashed in, the final leftover weight is always $|\text{sum}(A) - \text{sum}(B)|$ for *some* way of splitting the stones into two piles A and B (every smash just flips a sign inside a running total). So minimizing the final stone is *exactly* **Minimum Subset Sum Difference (A.5)** applied to the stones. We solve it here with the same add/subtract recursion style used in Target Sum (A.7), for practice with the three-step ladder.

Step 1 – Plain Recursion

```
// f(i, diff) : diff = (sum assigned to pile A) - (sum assigned to pile B) so far
int f(int i, int diff, vector<int>& stones) {
    if (i == stones.size()) return abs(diff);

    int putInA = f(i + 1, diff + stones[i], stones);
    int putInB = f(i + 1, diff - stones[i], stones);
    return min(putInA, putInB);
}

int lastStoneWeightII(vector<int>& stones) {
    return f(0, 0, stones);
}
```

Step 2 – Top-Down DP (Memoization)

```
int f(int i, int diff, vector<int>& stones, int total, vector<vector<int>>& dp) {
    if (i == stones.size()) return abs(diff);

    int idx = diff + total; // shift into a valid array index
    if (dp[i][idx] != -1) return dp[i][idx];

    int putInA = f(i + 1, diff + stones[i], stones, total, dp);
    int putInB = f(i + 1, diff - stones[i], stones, total, dp);

    return dp[i][idx] = min(putInA, putInB);
}

int lastStoneWeightII(vector<int>& stones) {
    int total = accumulate(stones.begin(), stones.end(), 0);
    vector<vector<int>> dp(stones.size(), vector<int>(2 * total + 1, -1));
    return f(0, 0, stones, total, dp);
}
```

Complexity: $O(n * \text{total})$ time and space.

Step 3 – Bottom-Up DP (via Subset Sum, same as A.5)

```
int lastStoneWeightII(vector<int>& stones) {
    int total = accumulate(stones.begin(), stones.end(), 0);
```

```

int n = stones.size();

vector<vector<bool>> dp(n + 1, vector<bool>(total + 1, false));
for (int i = 0; i <= n; i++) dp[i][0] = true;

for (int i = 1; i <= n; i++)
    for (int s = 1; s <= total; s++) {
        dp[i][s] = dp[i - 1][s];
        if (stones[i - 1] <= s)
            dp[i][s] = dp[i][s] || dp[i - 1][s - stones[i - 1]];
    }

int best = INT_MAX;
for (int s = 0; s <= total / 2; s++)
    if (dp[n][s]) best = min(best, total - 2 * s);    // closest achievable half

return best;
}

```

Complexity: $O(n * \text{total})$ time and space – structurally identical to A.5.

Alternative Optimal Approach (Maximize Achievable Sum – cleanest production version)

Instead of a boolean “is sum s reachable?” table, we can directly compute the **maximum achievable sum that does not exceed $\text{total}/2$** – this is exactly 0/1 Knapsack’s original “maximize value within capacity” recurrence from A.1, where each stone’s weight and value are both **stone** itself and the capacity is $\text{total}/2$.

```

int lastStoneWeightII(vector<int>& stones) {
    int total = accumulate(stones.begin(), stones.end(), 0);
    int target = total / 2;

    vector<int> dp(target + 1, 0);    // dp[j] = max achievable sum <= j

    for (int stone : stones) {
        for (int j = target; j >= stone; j--) {    // 0/1 knapsack: right to left (each stone used once)
            dp[j] = max(dp[j], dp[j - stone] + stone);
        }
    }

    return total - 2 * dp[target];
}

```

Since $\text{dp}[\text{target}]$ is the closest achievable pile sum to $\text{total}/2$, the other pile has sum $\text{total} - \text{dp}[\text{target}]$, and the final stone weight is their difference: $(\text{total} - \text{dp}[\text{target}]) - \text{dp}[\text{target}] = \text{total} - 2 * \text{dp}[\text{target}]$. This version skips the intermediate boolean table entirely and is the version most commonly seen in accepted submissions.

Complexity: $O(n * \text{target})$ time, $O(\text{target})$ space – same complexity class as the boolean version, but a single pass with no wasted boolean bookkeeping.

C.5 Last Stone Weight I (LeetCode 1046)

Link: <https://leetcode.com/problems/last-stone-weight/>

Problem: Same smashing rule as above, but this time you **must always smash the two currently heaviest stones** (no free choice of pairing). Return the weight of the final leftover stone (0 if none).

Understanding: Unlike Last Stone Weight II, here the smashing rule is *fixed*, not chosen – so there is nothing to optimize over and no overlapping sub-problems to memoize; this is a **greedy simulation problem, not a DP problem**. It is included here because it is commonly paired with Last Stone Weight II on problem sheets. Each step deterministically reduces to a smaller state, so a simple recursive simulation already runs efficiently:

```
// Recursive simulation: always smash the two heaviest stones (rule given by the problem)
int f(vector<int> stones) {
    sort(stones.begin(), stones.end());
    int n = stones.size();
    if (n == 0) return 0;
    if (n == 1) return stones[0];

    int y = stones[n - 1], x = stones[n - 2];
    vector<int> next(stones.begin(), stones.end() - 2);
    if (y != x) next.push_back(y - x);

    return f(next);           // exactly one next state -- nothing to memoize
}

int lastStoneWeight(vector<int>& stones) {
    return f(stones);
}
```

Optimal / practical implementation (max-heap simulation): re-sorting on every step is wasteful; a max-heap gives the two heaviest stones in $O(\log n)$ each round.

```
int lastStoneWeight(vector<int>& stones) {
    priority_queue<int> pq(stones.begin(), stones.end()); // max-heap
    while (pq.size() > 1) {
        int y = pq.top(); pq.pop();
        int x = pq.top(); pq.pop();
        if (y != x) pq.push(y - x);
    }
    return pq.empty() ? 0 : pq.top();
}
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

PART D – More Longest Increasing Subsequence Family Problems

D.1 Longest Continuous Increasing Subsequence (LeetCode 674)

Link: <https://leetcode.com/problems/longest-continuous-increasing-subsequence/>

Problem: Find the length of the longest **contiguous (no gaps allowed)** strictly increasing run in the array.

Understanding: This is a much simpler cousin of LIS – because the run must be contiguous, we don't need any nested loop comparing against *all* previous indices; we only ever compare `nums[i]` against its **immediate** neighbor `nums[i-1]`. A single linear pass with a running counter suffices.

```
int findLengthOfLCIS(vector<int>& nums) {
    int n = nums.size();
    int ans = 1, curr = 1;

    for (int i = 1; i < n; i++) {
        if (nums[i] > nums[i - 1]) curr++;    // extend the current run
        else curr = 1;                       // run breaks, restart
        ans = max(ans, curr);
    }
    return ans;
}
```

Complexity: $O(n)$ time, $O(1)$ space – no `dp[]` array is even needed since only the previous state matters.

D.2 Increasing Triplet Subsequence (LeetCode 334)

Link: <https://leetcode.com/problems/increasing-triplet-subsequence/>

Problem: Determine if there exist three indices $i < j < k$ such that $\text{nums}[i] < \text{nums}[j] < \text{nums}[k]$ (i.e. does an increasing subsequence of length **exactly 3** exist).

Understanding: This is a special case of the $O(n \log n)$ LIS technique (B.1, Step 4) capped at length 3: we only need to track the smallest “tail” for a subsequence of length 1 (**first**) and length 2 (**second**); the moment some element beats both, we have found our length-3 subsequence.

```
bool increasingTriplet(vector<int>& nums) {
    int first = INT_MAX, second = INT_MAX;

    for (int x : nums) {
        if (x <= first) {
            first = x;                // smallest seen so far -> best start of a chain
        } else if (x <= second) {
            second = x;              // smallest valid "middle" value found so far
        } else {
            return true;             // x is bigger than some valid first AND second
        }
    }
    return false;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

D.3 Wiggle Subsequence (LeetCode 376)

Link: <https://leetcode.com/problems/wiggle-subsequence/>

Problem: Find the length of the longest subsequence that strictly alternates between going up and going down (a “zigzag”).

Understanding: Track two running states as we scan left to right:

- **up** = length of the longest wiggle subsequence ending at the current position where the **last move went up**.
- **down** = length of the longest wiggle subsequence ending at the current position where the **last move went down**.

If the next number is bigger than the previous, it can extend a “down” sequence by going up: $up = down + 1$. Symmetrically for a decrease. Equal values change nothing (can’t wiggle on a flat step).

```
int wiggleMaxLength(vector<int>& nums) {
    int n = nums.size();
    if (n < 2) return n;

    int up = 1, down = 1;
    for (int i = 1; i < n; i++) {
        if (nums[i] > nums[i - 1]) up = down + 1;           // extend a down-run by going up
        else if (nums[i] < nums[i - 1]) down = up + 1;       // extend an up-run by going down
        // equal values: no change to either
    }
    return max(up, down);
}
```

Complexity: $O(n)$ time, $O(1)$ space.

D.4 Minimum Number of Removals to Make Mountain Array (LeetCode 1671)

Link: <https://leetcode.com/problems/minimum-number-of-removals-to-make-mountain-array/>

Problem: A “mountain array” strictly increases then strictly decreases (same shape as B.6’s bitonic subsequence, but here it must include **at least one element on each side** of the peak). Find the minimum number of elements to remove so the remaining array is a mountain.

Understanding: Directly reuses the **Longest Bitonic Subsequence** machinery from B.6. If the longest valid “mountain-shaped” subsequence (with a real climb and a real descent on both sides of some peak) has length **best**, then the minimum removals is simply $n - \text{best}$. The only extra care needed is to **reject peaks at the very start or end** of the array, since a mountain needs at least one element strictly before and after the peak.

```
int minimumMountainRemovals(vector<int>& nums) {
    int n = nums.size();
    vector<int> inc(n, 1), dec(n, 1);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (nums[j] < nums[i]) inc[i] = max(inc[i], 1 + inc[j]);

    for (int i = n - 1; i >= 0; i--)
        for (int j = n - 1; j > i; j--)
            if (nums[j] < nums[i]) dec[i] = max(dec[i], 1 + dec[j]);

    int best = 0;
    for (int i = 0; i < n; i++) {
        if (inc[i] > 1 && dec[i] > 1) // must have a real climb AND a real descent
            best = max(best, inc[i] + dec[i] - 1);
    }
    return n - best;
}
```

Complexity: $O(n^2)$ time, $O(n)$ space.

D.5 Best Team With No Conflicts (LeetCode 1626)

Link: <https://leetcode.com/problems/best-team-with-no-conflicts/>

Problem: Given player `ages[]` and `scores[]`, build a team maximizing total score such that no player with a **strictly higher score** is **younger** than a teammate (conflicts are forbidden). Find the maximum possible total score.

Understanding: This is **Maximum Sum Increasing Subsequence (B.8)** wearing a disguise, combined with the same sorting trick used in Russian Doll Envelopes (B.10): sort players by age, breaking ties by score ascending. After sorting, a valid team corresponds exactly to a subsequence whose **scores are non-decreasing** as we scan left to right (older-or-equal-age players must not have a lower score than an earlier, younger-or-equal-age teammate). Maximize the *sum* of such a subsequence, like B.8.

```
int bestTeamScore(vector<int>& scores, vector<int>& ages) {
    int n = scores.size();
    vector<pair<int,int>> players(n);
    for (int i = 0; i < n; i++) players[i] = {ages[i], scores[i]};

    sort(players.begin(), players.end());    // sort by age, ties broken by score ascending

    vector<int> dp(n);
    for (int i = 0; i < n; i++) dp[i] = players[i].second;    // team of just this player

    int ans = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (players[j].second <= players[i].second)    // no conflict, scores non-decreasing
                dp[i] = max(dp[i], dp[j] + players[i].second);
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

Complexity: $O(n^2)$ time, $O(n)$ space – identical skeleton to Maximum Sum Increasing Subsequence, just with a custom sort first.

PART E – Classic 2-D / Interval DP: LCS and Matrix Chain Multiplication

These two problems are included alongside Knapsack and LIS because they are the other two “textbook pillars” of DP, and they follow the exact same three-step ladder.

E.1 Longest Common Subsequence (LCS)

Problem: Given two strings `text1` and `text2`, find the length of their longest subsequence common to both (characters need not be contiguous in either string, but relative order must be preserved in both).

Understanding

Compare the strings from the back. Let $f(i, j)$ = LCS length of `text1[0..i-1]` and `text2[0..j-1]` (1-indexed lengths for convenience):

- If `text1[i-1] == text2[j-1]` – this matching character **must** be part of the LCS, so include it and recurse on both remaining prefixes: $f(i, j) = 1 + f(i-1, j-1)$.
- Otherwise, the last character of at least one string is *not* in the LCS, so try dropping each one and take the better result: $f(i, j) = \max(f(i-1, j), f(i, j-1))$.

State: (i, j) = how much of each string is still being considered. **Base case:** $f(0, j) = f(i, 0) = 0$ (an empty string has no common subsequence with anything).

Step 1 – Plain Recursion

```
int f(int i, int j, string& text1, string& text2) {
    if (i == 0 || j == 0) return 0;           // base case: one string is empty

    if (text1[i - 1] == text2[j - 1])
        return 1 + f(i - 1, j - 1, text1, text2);    // matching char, must use it

    return max(f(i - 1, j, text1, text2), f(i, j - 1, text1, text2)); // try dropping either char
}

int longestCommonSubsequence(string text1, string text2) {
    return f(text1.size(), text2.size(), text1, text2);
}
```

Step 2 – Top-Down DP (Memoization)

```
int f(int i, int j, string& text1, string& text2, vector<vector<int>>& dp) {
    if (i == 0 || j == 0) return 0;
    if (dp[i][j] != -1) return dp[i][j];

    if (text1[i - 1] == text2[j - 1])
        return dp[i][j] = 1 + f(i - 1, j - 1, text1, text2, dp);

    return dp[i][j] = max(f(i - 1, j, text1, text2, dp), f(i, j - 1, text1, text2, dp));
}

int longestCommonSubsequence(string text1, string text2) {
    int n = text1.size(), m = text2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, -1));
}
```

```
    return f(n, m, text1, text2, dp);  
}
```

Complexity: $O(n * m)$ time and space.

Step 3 – Bottom-Up DP (Tabulation)

```
int longestCommonSubsequence(string text1, string text2) {  
    int n = text1.size(), m = text2.size();  
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));    // row/col 0 already correctly 0  
  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= m; j++) {  
            if (text1[i - 1] == text2[j - 1])  
                dp[i][j] = 1 + dp[i - 1][j - 1];  
            else  
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);  
        }  
    }  
    return dp[n][m];  
}
```

Complexity: $O(n * m)$ time, $O(n * m)$ space (can be compressed to $O(\min(n, m))$ using two rolling rows, since row i only needs row $i-1$).

E.2 Matrix Chain Multiplication (MCM)

Problem: Given the dimensions of n matrices to multiply in a fixed order (matrix i has dimensions $p[i-1] \times p[i]$), find the **minimum number of scalar multiplications** needed to compute the full product, by choosing the optimal placement of parentheses (matrix multiplication is associative, so grouping changes the cost).

Understanding

Unlike Knapsack/LIS, the state here is a **range** (i, j) , representing “the cost of optimally multiplying matrices i through j ”. We try every possible split point k where the chain is cut into two multiplied sub-chains $(i..k)$ and $(k+1..j)$, then combine the two results with one final multiplication of size $p[i-1] \times p[k] \times p[j]$:

$$f(i, j) = \min \text{ over } k \text{ in } [i, j-1] \text{ of } (f(i, k) + f(k+1, j) + p[i-1]*p[k]*p[j])$$

Base case: $f(i, i) = 0$ (a single matrix needs no multiplication).

Step 1 – Plain Recursion

```
int f(int i, int j, vector<int>& p) {
    if (i == j) return 0;                // single matrix: no cost

    int minCost = INT_MAX;
    for (int k = i; k < j; k++) {
        int cost = f(i, k, p) + f(k + 1, j, p) + p[i - 1] * p[k] * p[j];
        minCost = min(minCost, cost);
    }
    return minCost;
}

int matrixChainOrder(vector<int>& p) {
    int n = p.size() - 1;                // number of matrices
    return f(1, n, p);
}
```

Why it is slow: the same range (i, j) is revisited through many different split paths – exponential time.

Step 2 – Top-Down DP (Memoization)

```
int f(int i, int j, vector<int>& p, vector<vector<int>>& dp) {
    if (i == j) return 0;
    if (dp[i][j] != -1) return dp[i][j];

    int minCost = INT_MAX;
    for (int k = i; k < j; k++) {
        int cost = f(i, k, p, dp) + f(k + 1, j, p, dp) + p[i - 1] * p[k] * p[j];
        minCost = min(minCost, cost);
    }
    return dp[i][j] = minCost;
}

int matrixChainOrder(vector<int>& p) {
    int n = p.size() - 1;
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, -1));
}
```

```

    return f(1, n, p, dp);
}

```

Complexity: $O(n^3)$ time (n^2 states, each doing an $O(n)$ split loop), $O(n^2)$ space.

Step 3 – Bottom-Up DP (Tabulation)

Because $f(i, j)$ depends on **smaller ranges** (i, k) and $(k+1, j)$ (both strictly shorter than (i, j)), we must fill the table in order of **increasing chain length**, not increasing i – this is the key difference from Knapsack/LIS tabulation.

```

int matrixChainOrder(vector<int>& p) {
    int n = p.size() - 1;
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));

    for (int len = 2; len <= n; len++) {           // chain length, smallest first
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j];
                dp[i][j] = min(dp[i][j], cost);
            }
        }
    }
    return dp[1][n];
}

```

Complexity: $O(n^3)$ time, $O(n^2)$ space.

PART F – Even More Knapsack Family Problems

F.1 House Robber (LeetCode 198)

Link: <https://leetcode.com/problems/house-robber/>

Problem: Given the money stashed in each house along a street, find the maximum amount you can rob such that **no two adjacent houses** are robbed.

Understanding: The simplest possible “take / not take” DP with an adjacency constraint instead of a weight constraint – it is the engine hiding inside Delete and Earn (B.12). $dp[i]$ = best amount robbable considering houses $0..i$: either skip house i ($dp[i-1]$) or rob it and add to the best up to two houses back ($dp[i-2] + nums[i]$).

```
int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];

    vector<int> dp(n);
    dp[0] = nums[0];
    dp[1] = max(nums[0], nums[1]);

    for (int i = 2; i < n; i++)
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i]);    // skip i, or take i + best up to i-2

    return dp[n - 1];
}
```

Complexity: $O(n)$ time, $O(n)$ space (easily compressible to $O(1)$ using two rolling variables).

F.2 House Robber II (LeetCode 213)

Link: <https://leetcode.com/problems/house-robber-ii/>

Problem: Same as House Robber, but the houses are arranged in a **circle** – the first and last houses are now also adjacent.

Understanding: The only new difficulty is that we can't decide about *both* the first and last house independently anymore. The standard trick: since robbing both endpoints is never simultaneously allowed, run plain House Robber **twice** – once over houses $[0, n-2]$ (excluding the last house) and once over $[1, n-1]$ (excluding the first house) – and take the better of the two answers. This guarantees the circular adjacency is never violated.

```
int robLine(vector<int>& nums, int start, int end) {
    int prev2 = 0, prev1 = 0;
    for (int i = start; i <= end; i++) {
        int curr = max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 1) return nums[0];
    return max(robLine(nums, 0, n - 2), robLine(nums, 1, n - 1));
}
```

Complexity: $O(n)$ time, $O(1)$ space.

F.3 Perfect Squares (LeetCode 279)

Link: <https://leetcode.com/problems/perfect-squares/>

Problem: Given an integer n , find the **least number of perfect square numbers** (1, 4, 9, 16, ...) that sum to n .

Understanding: This is **Coin Change (C.1)** where the “coin denominations” are the perfect squares $\leq n$, each reusable any number of times – an Unbounded Knapsack minimizing count.

```
int numSquares(int n) {
    vector<int> dp(n + 1, INT_MAX);
    dp[0] = 0;

    for (int i = 1; i <= n; i++) {
        for (int k = 1; k * k <= i; k++) {
            dp[i] = min(dp[i], dp[i - k * k] + 1);    // use one more copy of square k*k
        }
    }
    return dp[n];
}
```

Complexity: $O(n * \sqrt{n})$ time, $O(n)$ space.

F.4 Combination Sum IV (LeetCode 377)

Link: <https://leetcode.com/problems/combination-sum-iv/>

Problem: Given distinct positive integers `nums` and a `target`, count the number of possible **ordered sequences** that add up to `target` (unlike Coin Change II, [1, 2] and [2, 1] are counted as *different* results here).

Understanding: Same unbounded-reuse recurrence as **Coin Change II (C.2)**, but the loop order is **flipped**: put `target` on the *outside* and `nums` on the *inside*. This re-explores every number at every amount independently of order, so each distinct sequence (permutation) gets counted separately instead of being collapsed into one combination.

```
int combinationSum4(vector<int>& nums, int target) {
    vector<unsigned int> dp(target + 1, 0);
    dp[0] = 1;                                // one way to reach 0: the empty sequence

    for (int t = 1; t <= target; t++) {
        for (int num : nums) {
            if (num <= t) dp[t] += dp[t - num];
        }
    }
    return dp[target];
}
```

Side-by-side with Coin Change II: counting **combinations** (order doesn't matter) -> loop `coins` outside, `amount` inside. Counting **sequences/permutations** (order matters) -> loop `amount` outside, `nums` inside. This exact loop-order distinction is one of the most common DP interview gotchas.

Complexity: $O(\text{target} * n)$ time, $O(\text{target})$ space.

F.5 Word Break (LeetCode 139)

Link: <https://leetcode.com/problems/word-break/>

Problem: Given a string s and a dictionary of words (each word reusable any number of times), determine whether s can be segmented into a space-separated sequence of one or more dictionary words.

Understanding: An Unbounded-Knapsack-flavored string DP. $dp[i] = \text{true}$ if the prefix $s[0..i)$ can be fully segmented using dictionary words. To compute $dp[i]$, look for **some** earlier breakpoint $j < i$ such that $dp[j]$ is already true **and** the remaining chunk $s[j..i)$ is itself a dictionary word.

```
bool wordBreak(string s, vector<string>& wordDict) {
    unordered_set<string> dict(wordDict.begin(), wordDict.end());
    int n = s.size();
    vector<bool> dp(n + 1, false);
    dp[0] = true;                                // empty prefix is trivially breakable

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.count(s.substr(j, i - j))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[n];
}
```

Complexity: $O(n^2)$ time (ignoring substring-hashing cost), $O(n)$ space.

PART G – Even More LIS Family Problems

G.1 Longest Increasing Path in a Matrix (LeetCode 329)

Link: <https://leetcode.com/problems/longest-increasing-path-in-a-matrix/>

Problem: Given an $m \times n$ integer matrix, find the length of the longest strictly increasing path, where from each cell you may move to any of its **4 adjacent cells** (up/down/left/right).

Understanding: LIS generalized from a 1-D array to a 2-D grid **graph**: instead of only looking “backwards” along one line, each cell can extend the path from any of its 4 neighbors that hold a smaller value. This naturally maps to **top-down DP (DFS + memoization)**, exactly the same shape as B.1’s Step 2, but the “previous states” are graph neighbors instead of `nums[0..i-1]`.

```
int dx[4] = {0, 0, 1, -1}, dy[4] = {1, -1, 0, 0};

int dfs(int i, int j, vector<vector<int>>& matrix, vector<vector<int>>& dp) {
    if (dp[i][j] != -1) return dp[i][j];

    int best = 1;
    for (int d = 0; d < 4; d++) {
        int ni = i + dx[d], nj = j + dy[d];
        if (ni >= 0 && ni < (int)matrix.size() && nj >= 0 && nj < (int)matrix[0].size()
            && matrix[ni][nj] > matrix[i][j]) {
            best = max(best, 1 + dfs(ni, nj, matrix, dp));
        }
    }
    return dp[i][j] = best;
}

int longestIncreasingPath(vector<vector<int>>& matrix) {
    int m = matrix.size(), n = matrix[0].size();
    vector<vector<int>> dp(m, vector<int>(n, -1));

    int ans = 1;
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            ans = max(ans, dfs(i, j, matrix, dp));
    return ans;
}
```

Why bottom-up tabulation is awkward here: on a 1-D array we can tabulate left to right because `nums[j]` for $j < i$ is always “already done”. On a grid there is no single sweep order that guarantees all smaller neighbors are computed first (a path can increase in any direction), so **top-down memoization is the natural and standard solution** for this particular variant.

Complexity: $O(m * n)$ time (each cell’s DFS is computed once thanks to memoization), $O(m * n)$ space.

G.2 Count Number of Teams (LeetCode 1395)

Link: <https://leetcode.com/problems/count-number-of-teams/>

Problem: Given soldier `rating[]`, count the number of ways to choose 3 soldiers $i < j < k$ such that either $rating[i] < rating[j] < rating[k]$ (increasing triplet) or $rating[i] > rating[j] > rating[k]$ (decreasing triplet).

Understanding: A **counting** cousin of the $O(n^2)$ LIS comparison style (rather than optimizing a length, we count qualifying triplets). Treat each index j as the **middle** soldier of the triplet: count how many soldiers to its left are smaller (`leftSmaller`) and how many are larger (`leftLarger`); do the same to the right (`rightSmaller`, `rightLarger`). Every increasing triplet through j needs a smaller soldier on the left and a larger one on the right; every decreasing triplet needs the opposite.

```
int numTeams(vector<int>& rating) {
    int n = rating.size();
    int ans = 0;

    for (int j = 0; j < n; j++) {
        int leftSmaller = 0, leftLarger = 0, rightSmaller = 0, rightLarger = 0;

        for (int i = 0; i < j; i++) {
            if (rating[i] < rating[j]) leftSmaller++;
            else if (rating[i] > rating[j]) leftLarger++;
        }
        for (int k = j + 1; k < n; k++) {
            if (rating[k] < rating[j]) rightSmaller++;
            else if (rating[k] > rating[j]) rightLarger++;
        }

        ans += leftSmaller * rightLarger;    // increasing triplet centered at j
        ans += leftLarger * rightSmaller;    // decreasing triplet centered at j
    }
    return ans;
}
```

Complexity: $O(n^2)$ time, $O(1)$ extra space – the same nested-comparison flavor as $O(n^2)$ LIS, generalized from “optimize a length” to “count qualifying triples”.

Summary Table

#	Problem	Core Pattern	Time Complexity
A.1	0/1 Knapsack	take / not take	$O(n \cdot W)$
A.2	Subset Sum	take / not take (boolean)	$O(n \cdot \text{sum})$
A.3	Equal Sum Partition	Subset Sum, target = total/2	$O(n \cdot \text{total})$
A.4	Count Subsets with Sum	take / not take (count)	$O(n \cdot \text{sum})$
A.5	Min Subset Sum Difference	Subset Sum + closest half	$O(n \cdot \text{total})$
A.6	Count Subsets with Difference	reduces to A.4	$O(n \cdot \text{total})$
A.7	Target Sum	reduces to A.6	$O(n \cdot \text{total})$
B.1	Longest Increasing Subsequence	dp[i] ending at i	$O(n^2) / O(n \log n)$
B.2	Print LIS	LIS + parent[] backtracking	$O(n^2)$
B.3	Number of LIS	LIS + count tracking	$O(n^2)$
B.4	Longest Non-Decreasing Subsequence	LIS with $\leq$	$O(n^2)$
B.5	Circular LIS	LIS on doubled array	$O(n^3)$
B.6	Longest Bitonic Subsequence	LIS forward + LIS backward	$O(n^2)$
B.7	Longest Arithmetic Subsequence	dp[i][diff] via hashmap	$O(n^2)$
B.8	Max Sum Increasing Subsequence	LIS with sum instead of count	$O(n^2)$
B.9	Largest Divisible Subset	LIS with % instead of <	$O(n^2)$
B.10	Russian Doll Envelopes	sort + LIS ($O(n \log n)$)	$O(n \log n)$
B.11	Max Length of Pair Chain	LIS-style DP or greedy	$O(n^2) / O(n \log n)$
B.12	Delete and Earn	House Robber (take/not take)	$O(n + \text{maxVal})$
B.13	Longest String Chain	sort by length + LIS	$O(n^2 \cdot L)$
C.1	Coin Change	unbounded take/not take (min)	$O(n \cdot \text{amount})$
C.2	Coin Change II	unbounded take/not take (count)	$O(n \cdot \text{amount})$
C.3	Ones and Zeroes	0/1 knapsack, 2 capacities	$O(\text{len.m.n})$
C.4	Last Stone Weight II	reduces to A.5 (2 approaches shown)	$O(n \cdot \text{total})$
C.5	Last Stone Weight I	greedy max-heap (not DP)	$O(n \log n)$
D.1	Longest Continuous Increasing Subsequence	linear scan	$O(n)$
D.2	Increasing Triplet Subsequence	capped $O(n \log n)$ LIS trick	$O(n)$
D.3	Wiggle Subsequence	up/down running state	$O(n)$

#	Problem	Core Pattern	Time Complexity
D.4	Min Removals for Mountain Array	reuses B.6 (bitonic)	$O(n^2)$
D.5	Best Team With No Conflicts	sort + B.8 (max sum LIS)	$O(n^2)$
E.1	Longest Common Subsequence	dp[i][j] on two strings	$O(n.m)$
E.2	Matrix Chain Multiplication	interval DP, split point k	$O(n^3)$
F.1	House Robber	take/not take, adjacency	$O(n)$
F.2	House Robber II	House Robber run twice (circle)	$O(n)$
F.3	Perfect Squares	unbounded knapsack (min count)	$O(n.\sqrt{n})$
F.4	Combination Sum IV	unbounded, counts sequences	$O(\text{target}.n)$
F.5	Word Break	unbounded knapsack on strings	$O(n^2)$
G.1	Longest Increasing Path in Matrix	LIS generalized to a grid graph	$O(m.n)$
G.2	Count Number of Teams	counting version of $O(n^2)$ LIS	$O(n^2)$

Conclusion

Every problem in this lab collapses into one of just **three skeletons**:

- **“Take or don’t take”** (0/1 / Unbounded Knapsack skeleton) – used whenever we are choosing a subset (or repeated selection) of items under some sum/count/difference/capacity/adjacency constraint. The 0/1 vs. Unbounded distinction shows up as a one-line change: move to index $i-1$ after taking (0/1) vs. stay at index i (unbounded) – equivalently, loop capacity right-to-left vs. left-to-right in the space-optimized tabulation. Counting **combinations** vs. counting **ordered sequences** is likewise a one-line change: loop items outside amount (combinations) vs. amount outside items (sequences) – see Coin Change II vs. Combination Sum IV.
- **“Extend the best subsequence ending here”** (LIS skeleton) – used whenever we are building an ordered chain where each next element must satisfy some comparison against the previous one ($<$, $\leq$, $\%$, “insert one letter”, non-decreasing score, grid-neighbor value, etc.), or when counting such chains/triplets instead of optimizing their length.
- **“Split the range and combine”** (Interval DP skeleton, seen in Matrix Chain Multiplication) and its close cousin **“align two sequences position by position”** (2-D skeleton, seen in LCS) – used whenever the state is a *range* or a *pair of positions* rather than a single index, and the answer is built from smaller ranges/prefixes.

Once the skeleton is recognized, solving a “new” problem is really just: **(1)** identify the state, **(2)** identify the choices, **(3)** write the recursion, **(4)** memoize it, **(5)** convert to tabulation (paying attention to the correct fill order – by index for Knapsack/LIS, by increasing length for interval DP, and noting that some grid-shaped LIS variants like G.1 only tabulate cleanly top-down), and, where possible, **(6)** optimize further (space compression, greedy, or $O(n \log n)$ techniques).